VOLUME-FREE ALGORITHMIC TRADING PLATFORM

# Strategy & Analysis Playbook 22 OHLC-Only Strategies

A structural reference built entirely from Open/High/Low/Close and timestamps — no volume, no order book, no delta. Two strategy sets, kept separate: a 16-strategy regime-gated structural blueprint (A–H), and a six-strategy classic core (S1–S6). Every strategy stands on its own card: premise, regime gate, trigger, stop, target, and runnable Python-style pseudocode.

---

| STRATEGIES | FAMILIES | DATA REQUIRED | SCOPE |
|---|---|---|---|
| **22** | **9** | **OHLC + timestamp** | **Paper trading / research** |

# Contents

22 strategies across 9 structural families. Each keeps its own card — nothing below is merged or averaged across strategies.


**TREND CONTINUATION**

- **A1** Trend Pullback to Dynamic Support
- **A2** Break-of-Structure Continuation
- **A3** Compression Flag Breakout Inside Trend

**BREAKOUT & EXPANSION**

- **B1** N-Bar Donchian Breakout
- **B2** Squeeze Release (BB-in-Keltner)
- **B3** Opening-Range Breakout

**MEAN REVERSION / RANGE FADE**

- **C1** Range Fade at Band Extremes
- **C2** Z-Score Reversion to Anchored TWAP

**STRUCTURAL REVERSAL**

- **D1** Failed Breakout / Sweep Reversal
- **D2** CHoCH Exhaustion Reversal

**MOMENTUM THRUST**

- **E1** Thrust Bar Follow-Through

**RELATIVE VALUE (PAIRS)**

- **F1** Cointegrated Pairs Z-Spread
- **F2** Beta-Hedged Residual Reversion

**TIME & CALENDAR**

- **G1** Session / Time-of-Day Bias
- **G2** Seasonality (Turn-of-Month / Day-of-Week)

**ADAPTIVE RANGE LATTICE**

- **H1** ATR-Spaced Range Lattice

**CORE SIX (CLASSIC FORMULATION)**

- **S1** Trend + Pullback (EMA Stack + RSI)
- **S2** Volatility Breakout (Squeeze → Expansion)
- **S3** Mean-Reversion in Range (RSI + Bollinger)
- **S4** Pure Market Structure / Price Action
- **S5** Supertrend + ADX Filter
- **S6** Multi-Timeframe Momentum Confluence

# Feature Library

Every strategy card below calls these shared, OHLC-only helper functions by name instead of redefining them. Implement once in M3 (feature layer); every strategy in M4 is then a pure function of these outputs plus the current regime label.

### `atr(bars, i, n=14)`

Wilder ATR. TR = max(H-L, |H-C[-1]|, |L-C[-1]|); ATR = RMA(TR, n).

### `ema(bars, i, n)`

Exponential moving average, alpha = 2/(n+1).

### `sma(values)`

Simple average of a list of values.

### `rsi(bars, i, n=14)`

Wilder RSI from average gain / average loss over n bars.

### `adx(bars, i, n=14)`

Wilder ADX: +DM/-DM -> DI+/DI- -> DX -> RMA(DX, n). Trend strength, 0-100.

### `efficiency_ratio(bars, i, n)`

Kaufman ER = |C - C[-n]| / sum(|C[k]-C[k-1]|). Straightness of the move, 0-1.

### `ols_r2(bars, i, n)`

R-squared of an OLS regression of Close on bar index over the last n bars.

### `variance_ratio(bars, i, q)`

Var(q-bar returns) / (q * Var(1-bar returns)). >1 trending, <1 mean-reverting.

### `hurst(bars, i, window)`

Hurst exponent via DFA on log returns. <0.5 mean-reverting, >0.5 trending.

### `swing_high / swing_low(bars, i)`

Fractal pivot: local extreme vs. k bars on each side (k=2 default).

### `market_state(bars, i)`

'bull' (HH/HL) | 'bear' (LH/LL) | 'chop', from the tracked pivot sequence.

### `structure_is(pattern, bars, i)`

True if the last two tracked swings match a named pattern, e.g. 'HH_HL'.

### `bollinger_bands(bars, i, n, k)`

SMA(n) +/- k*stdev(n); returns (lower, mid, upper).

### `bollinger_bandwidth(bars, i, n, k)`

(upper - lower) / mid, from bollinger_bands().

### `bbwp(bars, i, lookback)`

Percentile rank of current Bollinger Bandwidth over the lookback window.

### `keltner / squeeze_active(bars, i)`

EMA(20) +/- 1.5*ATR; squeeze = Bollinger Bands inside the Keltner Channel.

### `zscore(bars, i, n)`

(Close - SMA(n)) / stdev(n).

### `anchored_twap(bars, i)`

Mean(Close) since the session/anchor open -- the volume-free VWAP substitute.

### `supertrend(bars, i, m, n)`

ATR-trailing trend line; returns {value, direction}.

### `adf_pvalue(residual) / ols_residual(a, b, i)`

Cointegration residual and its ADF stationarity p-value, for pairs.

### `true_range(bars, i)`

Current-bar TR (see atr above), unsmoothed.

### `Order / PairOrder / Ladder`

Common return objects: entry, stop, targets (price, %, action), and optional kill/trail rules.

A · TREND CONTINUATION

# A1 Trend Pullback to Dynamic Support

Pullback into an established trend, then resumption.

### VALID REGIME

`ADX(14) > 25, ER(20) > 0.3`

### GATE (ALL MUST HOLD)

- state = HH/HL (bull) or LH/LL (bear)
- ADX(14) > 25
- ER(20) > 0.3
- $R^2(50) > 0.6$

### TRIGGER

Closed bar touches EMA20 ± 0.25×ATR; reversal bar (close in top 40% of range, low > prior swing low); RSI(14) > 40.

### ENTRY

Buy-stop 1 tick above trigger-bar high (confirmation, not a knife-catch limit).

### STOP

min(swing_low, trigger_bar_low) − 0.25×ATR. Reject the trade if that distance > 2×ATR.

### TAKE-PROFIT / EXIT

TP1 = 1R (close 50%, stop → breakeven) · TP2 = prior swing high or entry + 2.5×ATR · TP3 = runner.

### TRAIL / TIME STOP / KILL SWITCH

- Trail after TP1: Chandelier = highest_high(22) − 3×ATR, ratchet only
- Time stop: exit if no 1R progress in 10 bars; flat at session close if intraday

### PSEUDOCODE

```
def signal_A1(bars, i):
    state = market_state(bars, i)                # 'bull' | 'bear' | 'chop'
    if state == 'chop':
        return None
    if not (adx(bars, i, 14) > 25 and efficiency_ratio(bars, i, 20) > 0.3
            and ols_r2(bars, i, 50) > 0.6):
        return None

    atr_v, ema20 = atr(bars, i, 14), ema(bars, i, 20)
    c = bars[i]
    touched  = abs(c.close - ema20) <= 0.25 * atr_v
    reversal = close_position(c) >= 0.40 and c.low > swing_low(bars, i)
    if not (touched and reversal and rsi(bars, i, 14) > 40):
        return None

    direction = 'long' if state == 'bull' else 'short'
    entry = c.high + TICK if direction == 'long' else c.low - TICK
    raw_stop = (min(swing_low(bars, i), c.low) - 0.25 * atr_v if direction == 'long'
                else max(swing_high(bars, i), c.high) + 0.25 * atr_v)
    if abs(entry - raw_stop) > 2 * atr_v:
        return None                              # stop too wide -> skip

    r = abs(entry - raw_stop)
    tp1 = entry + r if direction == 'long' else entry - r
    tp2 = prior_swing(bars, i, direction) or (entry + 2.5*atr_v if direction=='long'
                                              else entry - 2.5*atr_v)

    return Order('A1', direction, 'stop', entry, raw_stop,
                 targets=[(tp1, 0.5, 'stop_to_breakeven'),
                          (tp2, 0.3, None),
                          (None, 0.2, 'trail_chandelier_22_3ATR')],
                 time_stop_bars=10, flat_at_session_close=True)
```

A · TREND CONTINUATION

# A2 Break-of-Structure Continuation

Internal structure breaks in the direction of the prevailing trend.

### VALID REGIME

`Same as A1, plus a confirmed internal pivot set`

### GATE (ALL MUST HOLD)

- Trend state confirmed (as A1)
- A tracked internal pivot exists
- ER rising

### TRIGGER

C > last_pivot_high + 0.1×ATR on a bar closing in its top 30%, no opposing HTF level within 1×ATR.

### ENTRY

Market on bar close, or a retest limit at the broken pivot (better R, lower fill rate — run both and compare).

### STOP

Below the broken pivot − 0.5×ATR, or 1.25×ATR if the structure level sits too close.

### TAKE-PROFIT / EXIT

Measured move = height of the prior swing projected from the break; else 2R fixed.

### TRAIL / TIME STOP / KILL SWITCH

- Kill: close back below the pivot within 3 bars → exit immediately (failed BOS)

### PSEUDOCODE

```
def signal_A2(bars, i):
    if market_state(bars, i) == 'chop':
        return None
    piv = last_pivot_high(bars, i)
    atr_v = atr(bars, i, 14)
    c = bars[i]
    broke = c.close > piv + 0.1*atr_v and close_position(c) >= 0.70
    if not (broke and rising(efficiency_ratio, bars, i, 20)):
        return None
    if opposing_htf_level_within(bars, i, 1*atr_v):
        return None

    entry = c.close                                # or retest limit at `piv`
    stop  = piv - 0.5*atr_v
    if abs(entry - stop) < 1.25*atr_v:
        stop = entry - 1.25*atr_v
    swing_h = prior_swing_height(bars, i)
    tp = entry + (swing_h or 2*abs(entry-stop))

    order = Order('A2', 'long', 'market', entry, stop,
                  targets=[(tp, 1.0, None)])
    order.kill_if = lambda b, j: b[j].close < piv     # within 3 bars of entry
    return order
```

A · TREND CONTINUATION

# A3 Compression Flag Breakout Inside Trend

A tight EMA-ribbon flag compresses mid-trend, then releases.

### VALID REGIME

`Trend state + ribbon_spread < 0.5×ATR + BBWP < 25th pct`

### GATE (ALL MUST HOLD)

- Trend state confirmed
- ribbon_spread(EMAs)/ATR < 0.5
- BBWP(100) < 25th percentile

### TRIGGER

Breakout candle closes outside the flag range with TR > 1.3×ATR, in the trend direction.

### ENTRY

Market/stop on the breakout candle close, in trend direction.

### STOP

Opposite flag boundary − 0.25×ATR (natural, efficient stop).

### TAKE-PROFIT / EXIT

Flag height projected; TP1 = 1.5R, TP2 = 3R or HTF swing.

### TRAIL / TIME STOP / KILL SWITCH

- Edge note: best R:R in the book, worst win rate — expect 35–45% hit rate at ~2.5R average

### PSEUDOCODE

```
def signal_A3(bars, i):
    if market_state(bars, i) == 'chop':
        return None
    atr_v = atr(bars, i, 14)
    ribbon = ribbon_spread(bars, i) / atr_v
    if not (ribbon < 0.5 and bbwp(bars, i, 100) < 25):
        return None

    flag_hi, flag_lo = flag_range(bars, i)
    c = bars[i]
    tr_v = true_range(bars, i)
    direction = trend_direction(bars, i)
    breakout = ((c.close > flag_hi) if direction == 'long' else (c.close < flag_lo))                and tr_v > 1.3*atr_v
    if not breakout:
        return None

    entry = c.close
    stop  = (flag_lo - 0.25*atr_v) if direction == 'long' else (flag_hi + 0.25*atr_v)
    flag_h = flag_hi - flag_lo
    r = abs(entry - stop)
    tp1 = entry + 1.5*r if direction=='long' else entry - 1.5*r
    tp2 = entry + flag_h if direction=='long' else entry - flag_h

    return Order('A3', direction, 'stop', entry, stop,
                 targets=[(tp1, 0.5, None), (tp2, 0.5, None)])
```

B · BREAKOUT & EXPANSION

# B1 N-Bar Donchian Breakout

Volatility compression resolves into a directional expansion.

### VALID REGIME

`ATRrk > 30 (expansion regime present)`

### GATE (ALL MUST HOLD)

- ATR percentile-rank(250) > 30
- No opposing HTF structure within 1×ATR

### TRIGGER

C > max(H,20) long / C < min(L,20) short; require ER(20) > 0.25 to filter range pokes.

### ENTRY

Stop order at the channel level + 1 tick; skip if the level was already tagged twice this week (liquidity grab — see D1).

### STOP

1.5×ATR from entry, or min(L,10) − 0.5×ATR, whichever is tighter.

### TAKE-PROFIT / EXIT

No fixed TP — pure trend-following, Chandelier trail 3×ATR. Fixed-TP variant: 3R.

### TRAIL / TIME STOP / KILL SWITCH

- Time stop: 20 bars; also flat before major session rollover

### PSEUDOCODE

```
def signal_B1(bars, i, n=20):
    if atr_percentile_rank(bars, i, 250) <= 30:
        return None
    hh, ll = max(b.high for b in bars[i-n:i]), min(b.low for b in bars[i-n:i])
    c = bars[i]
    if efficiency_ratio(bars, i, 20) <= 0.25:
        return None
    if c.close > hh:
        direction, level = 'long', hh
    elif c.close < ll:
        direction, level = 'short', ll
    else:
        return None
    if tagged_twice_this_week(bars, i, level):
        return None                                  # likely a liquidity grab, not a break

    atr_v = atr(bars, i, 14)
    entry = level + TICK if direction == 'long' else level - TICK
    stop_a = entry - 1.5*atr_v if direction=='long' else entry + 1.5*atr_v
    stop_b = (min(b.low for b in bars[i-10:i]) - 0.5*atr_v if direction=='long'
              else max(b.high for b in bars[i-10:i]) + 0.5*atr_v)
    stop = max(stop_a, stop_b) if direction=='long' else min(stop_a, stop_b)

    return Order('B1', direction, 'stop', entry, stop,
                 targets=[(None, 1.0, 'trail_chandelier_3ATR')],
                 time_stop_bars=20, flat_before_rollover=True)
```

# B2 Squeeze Release (BB-in-Keltner)

Bollinger Bands compress inside the Keltner Channel, then release.

### VALID REGIME

`Squeeze active ≥ 5 bars, BBWP < 20th pct`

### GATE (ALL MUST HOLD)

- squeeze = True for ≥ 5 bars
- BBWP(100) < 20th percentile

### TRIGGER

Squeeze releases (BB width > Keltner width) and the release bar closes in the top/bottom 25% of its range (direction-agnostic).

### ENTRY

Market on release-bar close; direction = bar's close position.

### STOP

Release-bar extreme − 0.25×ATR; alternative 1×ATR from entry (pick by backtest, not taste).

### TAKE-PROFIT / EXIT

2R fixed + 2×ATR extension target; take profit mechanically, don't get greedy.

### TRAIL / TIME STOP / KILL SWITCH

- Dead-trade rule: if the breakout doesn't achieve 0.5×ATR in 3 bars, scratch it

### PSEUDOCODE

```
def signal_B2(bars, i):
    if not (squeeze_active(bars, i, min_bars=5) and bbwp(bars, i, 100) < 20):
        return None
    if not squeeze_released(bars, i):
        return None

    c = bars[i]
    pos = close_position(c)
    if pos >= 0.75:
        direction = 'long'
    elif pos <= 0.25:
        direction = 'short'
    else:
        return None

    atr_v = atr(bars, i, 14)
    entry = c.close
    stop  = (c.low - 0.25*atr_v) if direction=='long' else (c.high + 0.25*atr_v)
    r = abs(entry - stop)
    tp = entry + 2*r if direction=='long' else entry - 2*r

    order = Order('B2', direction, 'market', entry, stop, targets=[(tp, 1.0, None)])
    order.scratch_if = lambda b, j: abs(b[j].close - entry) < 0.5*atr_v and (j - i) >= 3
    return order
```

# B3 Opening-Range Breakout

The first N minutes of the session set a range that the day breaks out of.

### VALID REGIME

`Requires session definition; OR = first 30–60 min`

### GATE (ALL MUST HOLD)

- Session/timezone defined
- ATR percentile-rank(250) > 40 (never trade ORB on dead-vol days)

### TRIGGER

C > OR_high + 0.15×OR_range (or symmetric short).

### ENTRY

Stop order at the OR extreme; on first break OR first retest, not both.

### STOP

OR_mid, or OR_low − 0.25×ATR, whichever is closer (naturally tight — that is the point).

### TAKE-PROFIT / EXIT

1×OR_range and 2×OR_range, scale 50/50; trail the remainder.

### TRAIL / TIME STOP / KILL SWITCH

- Time stop: hard flat 60–90 min before session close; never hold ORB overnight

### PSEUDOCODE

```
def signal_B3(bars, i, session):
    orr = opening_range(bars, session, minutes=30)      # (or_high, or_low)
    if atr_percentile_rank(bars, i, 250) <= 40:
        return None

    or_high, or_low = orr
    or_range = or_high - or_low
    c = bars[i]
    if c.close > or_high + 0.15*or_range:
        direction, level = 'long', or_high
    elif c.close < or_low - 0.15*or_range:
        direction, level = 'short', or_low
    else:
        return None
    if already_broke_and_retested_twice(bars, i, level):
        return None                                     # only first break OR first retest

    or_mid = (or_high + or_low) / 2
    atr_v = atr(bars, i, 14)
    stop_alt = or_low - 0.25*atr_v if direction=='long' else or_high + 0.25*atr_v
    stop = max(or_mid, stop_alt) if direction=='long' else min(or_mid, stop_alt)

    entry = level + TICK if direction=='long' else level - TICK
    tp1 = entry + or_range if direction=='long' else entry - or_range
    tp2 = entry + 2*or_range if direction=='long' else entry - 2*or_range

    return Order('B3', direction, 'stop', entry, stop,
                 targets=[(tp1, 0.5, None), (tp2, 0.5, 'trail_remainder')],
                 hard_flat_before_close_minutes=75)
```

C · MEAN REVERSION / RANGE FADE

# C1 Range Fade at Band Extremes

Fade a confirmed, non-trending range at its statistical extremes.

### VALID REGIME

`ADX < 20, ER < 0.25, range confirmed`

### GATE (ALL MUST HOLD)

- ADX(14) < 20
- ER(20) < 0.25
- Hurst < 0.45
- Range confirmed: ≥ 2 touches of each boundary

### TRIGGER

Price tags BB(20,2) extreme AND RSI(14) < 30 / > 70 AND a rejection bar prints (wick ≥ 50% of range).

### ENTRY

Market/limit at the confirmed rejection bar, fading toward the mean.

### STOP

Beyond the band extreme + 1×ATR, absolute cap 1.5×ATR (wide stops here = liquidation, not trading).

### TAKE-PROFIT / EXIT

TP1 = range mid (TWAP/POC proxy), TP2 = opposite band — never target beyond it.

### TRAIL / TIME STOP / KILL SWITCH

- Time stop: mandatory, 8–12 bars
- Regime kill: ADX crosses above 25 while in trade → exit at market, no debate

### PSEUDOCODE

```
def signal_C1(bars, i):
    if not (adx(bars, i, 14) < 20 and efficiency_ratio(bars, i, 20) < 0.25
            and hurst(bars, i, 256) < 0.45):
        return None
    if not range_confirmed(bars, i, min_touches=2):
        return None

    bb_lo, bb_mid, bb_hi = bollinger_bands(bars, i, 20, 2)
    c = bars[i]
    rsi_v = rsi(bars, i, 14)
    if c.low <= bb_lo and rsi_v < 30 and rejection_wick(c) >= 0.5:
        direction, extreme = 'long', bb_lo
    elif c.high >= bb_hi and rsi_v > 70 and rejection_wick(c) >= 0.5:
        direction, extreme = 'short', bb_hi
    else:
        return None

    atr_v = atr(bars, i, 14)
    stop = min(extreme - atr_v, extreme - 1.5*atr_v) if direction=='long'            else max(extreme + atr_v, extreme + 1.5*atr_v)
    entry = c.close

    order = Order('C1', direction, 'market', entry, stop,
                  targets=[(bb_mid, 0.6, None), (bb_hi if direction=='long' else bb_lo, 0.4, None)],
                  time_stop_bars=10)
    order.kill_if = lambda b, j: adx(b, j, 14) > 25
    return order
```

C · MEAN REVERSION / RANGE FADE

# C2 Z-Score Reversion to Anchored TWAP

Statistically stretched price mean-reverts to an anchored average.

### VALID REGIME

`|z(20)| > 2.0, VR(5) < 1, Hurst < 0.45`

### GATE (ALL MUST HOLD)

- |z(20)| > 2.0
- Variance Ratio(5) < 1
- Hurst < 0.45
- No BOS in the last 5 bars

### TRIGGER

z < −2 (long) / z > +2 (short) with stretch > 1.5×ATR from anchored TWAP.

### ENTRY

Limit at z = ±2, scaling to ±2.5 and ±3 (max 3 tranches, hard capped).

### STOP

z = ±3.5 or 2×ATR, whichever hits first — a hard stop, not a "wait for it" stop.

### TAKE-PROFIT / EXIT

z = 0 (the mean) for 70% of the position; remainder at z = ∓0.5.

### TRAIL / TIME STOP / KILL SWITCH

- Time stop: 15 bars; flatten at session end

### PSEUDOCODE

```
def signal_C2(bars, i):
    z = zscore(bars, i, 20)
    if abs(z) <= 2.0:
        return None
    if not (variance_ratio(bars, i, 5) < 1 and hurst(bars, i, 256) < 0.45):
        return None
    if bos_in_last_n_bars(bars, i, 5):
        return None

    atr_v = atr(bars, i, 14)
    stretch = abs(bars[i].close - anchored_twap(bars, i)) / atr_v
    if stretch <= 1.5:
        return None

    direction = 'long' if z < -2 else 'short'
    tranches = [(2.0, 0.34), (2.5, 0.33), (3.0, 0.33)]
    entries = [zscore_to_price(bars, i, lvl if direction=='long' else -lvl) for lvl, _ in tranches]

    stop = zscore_to_price(bars, i, 3.5 if direction=='short' else -3.5)
    stop_alt = entries[0] - 2*atr_v if direction=='long' else entries[0] + 2*atr_v
    stop = max(stop, stop_alt) if direction=='long' else min(stop, stop_alt)

    tp_main = anchored_twap(bars, i)                # z = 0
    tp_rest = zscore_to_price(bars, i, -0.5 if direction=='long' else 0.5)

    return Order('C2', direction, 'limit_ladder', entries, stop,
                 targets=[(tp_main, 0.7, None), (tp_rest, 0.3, None)],
                 time_stop_bars=15, flat_at_session_end=True)
```

D · STRUCTURAL REVERSAL

# D1 Failed Breakout / Sweep Reversal

Price sweeps a liquidity extreme, fails to hold, and reverses.

### VALID REGIME

`Instrument at ≥ 20-bar extreme, ATRrk > 50`

### GATE (ALL MUST HOLD)

- Price at a ≥ 20-bar high/low extreme
- ATR percentile-rank(250) > 50 (sweeps happen in nervous markets)

### TRIGGER

Price trades beyond the extreme but closes back inside within 2 bars, leaving a long wick; wait for a confirming close in the reversal direction.

### ENTRY

Market on the confirming close (never on the wick bar itself).

### STOP

Beyond the sweep extreme + 0.25×ATR — the invalidating level is objective and obvious.

### TAKE-PROFIT / EXIT

1R + opposite swing / range mid. Short-horizon pattern — don't hold for the whole move.

### TRAIL / TIME STOP / KILL SWITCH

- Volume-free substitute for order-flow "absorption" reads; captures the same event with price geometry only

### PSEUDOCODE

```
def signal_D1(bars, i):
    if atr_percentile_rank(bars, i, 250) <= 50:
        return None
    extreme_hi = max(b.high for b in bars[i-20:i])
    extreme_lo = min(b.low for b in bars[i-20:i])
    sweep = swept_and_closed_back(bars, i, extreme_hi, extreme_lo, within_bars=2)
    if not sweep:
        return None                                  # returns None if no wick-and-reclaim event

    direction, level = sweep.direction, sweep.level     # 'long' after a low-sweep, etc.
    confirmed = confirming_close(bars, i, direction)
    if not confirmed:
        return None

    atr_v = atr(bars, i, 14)
    entry = bars[i].close
    stop  = level - 0.25*atr_v if direction=='long' else level + 0.25*atr_v
    r = abs(entry - stop)
    tp1 = entry + r if direction=='long' else entry - r
    tp2 = opposite_swing_or_range_mid(bars, i, direction)

    return Order('D1', direction, 'market', entry, stop,
                 targets=[(tp1, 0.5, None), (tp2, 0.5, None)],
                 max_hold_bars=15)
```

D · STRUCTURAL REVERSAL

# D2 CHoCH Exhaustion Reversal

A stretched, decelerating trend confirms a change of character.

### VALID REGIME

`ADX was > 30, now declining 3+ bars`

### GATE (ALL MUST HOLD)

- ADX(14) was > 30, now declining ≥ 3 bars
- Price ≥ 2.5×ATR stretch from EMA20
- Momentum divergence: new swing extreme vs. lower-high/higher-low RSI

### TRIGGER

CHoCH confirmed — close beyond the last opposing pivot by ≥ 0.1×ATR.

### ENTRY

On the CHoCH close; add on the first pullback that holds the CHoCH level.

### STOP

Beyond the exhaustion extreme + 0.5×ATR.

### TAKE-PROFIT / EXIT

Origin of the last impulse (the structural magnet); scale at 2R and 3R.

### TRAIL / TIME STOP / KILL SWITCH

- Kill: price makes a new extreme after entry → the exhaustion read was wrong, exit

### PSEUDOCODE

```
def signal_D2(bars, i):
    if not adx_declining_from_above_30(bars, i, min_bars=3):
        return None
    atr_v = atr(bars, i, 14)
    stretch = abs(bars[i].close - ema(bars, i, 20)) / atr_v
    if stretch < 2.5 or not momentum_divergence(bars, i):
        return None

    piv = last_opposing_pivot(bars, i)
    choch = choch_confirmed(bars, i, piv, buffer=0.1*atr_v)
    if not choch:
        return None

    direction = choch.direction
    entry = bars[i].close
    exhaustion_extreme = choch.exhaustion_extreme
    stop = exhaustion_extreme + 0.5*atr_v if direction=='short' else exhaustion_extreme - 0.5*atr_v
    origin = last_impulse_origin(bars, i)
    r = abs(entry - stop)

    order = Order('D2', direction, 'market', entry, stop,
                  targets=[(origin, 0.5, None), (entry + 2*r if direction=='long' else entry-2*r, 0.25, None),
                           (entry + 3*r if direction=='long' else entry-3*r, 0.25, None)])
    order.add_on_pullback_to = choch.level
    order.kill_if = lambda b, j: new_extreme_since_entry(b, i, j, direction)
    return order
```

E · MOMENTUM THRUST

# E1 Thrust Bar Follow-Through

A single range-expansion bar signals immediate directional continuation.

### VALID REGIME

`TR > 1.8×ATR bar in the direction of state`

### GATE (ALL MUST HOLD)

- TR(bar) > 1.8×ATR, aligned with market state
- Session open or first 4 hours preferred

### TRIGGER

The thrust bar itself, on close — no pullback required.

### ENTRY

Stop order at the thrust-bar extreme + 1 tick, valid for the next 3 bars only (then expires).

### STOP

Opposite side of the thrust bar ∓ 0.25×ATR.

### TAKE-PROFIT / EXIT

1.5× and 2× the thrust bar's own range; trail the runner with 2.5×ATR.

### TRAIL / TIME STOP / KILL SWITCH

- Skip if the thrust is against an ER > 0.4 counter-trend, or occurs inside a BBWP < 20 squeeze (that's B2's trade)

### PSEUDOCODE

```
def signal_E1(bars, i):
    atr_v = atr(bars, i, 14)
    tr_v = true_range(bars, i)
    state = market_state(bars, i)
    if tr_v <= 1.8*atr_v or state == 'chop':
        return None

    c = bars[i]
    direction = 'long' if c.close > c.open else 'short'
    if direction != state:                          # thrust must align with prevailing state
        return None
    if efficiency_ratio(bars, i, 20) > 0.4 and direction != state:
        return None
    if bbwp(bars, i, 100) < 20:
        return None                                 # inside a squeeze -> that's B2's trade

    bar_range = c.high - c.low
    entry = c.high + TICK if direction=='long' else c.low - TICK
    stop  = (c.low - 0.25*atr_v) if direction=='long' else (c.high + 0.25*atr_v)
    tp1 = entry + 1.5*bar_range if direction=='long' else entry - 1.5*bar_range
    tp2 = entry + 2.0*bar_range if direction=='long' else entry - 2.0*bar_range

    return Order('E1', direction, 'stop', entry, stop,
                 targets=[(tp1, 0.5, None), (tp2, 0.3, None), (None, 0.2, 'trail_2.5ATR')],
                 order_expires_in_bars=3)
```

F · RELATIVE VALUE (PAIRS)

# F1 Cointegrated Pairs Z-Spread

A cointegrated spread between two instruments dislocates from equilibrium.

### VALID REGIME

`ADF(residual) p < 0.05, corr(60) > 0.7`

### GATE (ALL MUST HOLD)

- ADF test on residual: p < 0.05
- Rolling 60-bar correlation > 0.7
- Both legs liquid, rolling β stable within ±15%

### TRIGGER

|z_spread| > 2.0.

### ENTRY

Long the cheap leg / short the rich leg simultaneously — leg in, leg out, no naked-leg risk.

### STOP

|z| > 3.0, or a structural break in cointegration (ADF roll-off) — whichever first.

### TAKE-PROFIT / EXIT

z_spread = 0, scale 50% at z = ±0.5.

### TRAIL / TIME STOP / KILL SWITCH

- Needs margin on both legs, borrow/short access, and honest per-leg cost modeling — pairs die from costs, not signal quality

### PSEUDOCODE

```
def signal_F1(bars_a, bars_b, i):
    resid, beta = ols_residual(bars_a, bars_b, i, window=250)
    if adf_pvalue(resid) >= 0.05:
        return None
    if rolling_corr(bars_a, bars_b, i, 60) <= 0.7 or not beta_stable(beta, tol=0.15):
        return None

    z = zscore_series(resid, i, 60)
    if abs(z) <= 2.0:
        return None

    direction = 'long_a_short_b' if z < -2 else 'short_a_long_b'
    stop_z = 3.0
    tp_z   = 0.0

    return PairOrder('F1', direction,
                     leg_a=bars_a[i].close, leg_b=bars_b[i].close, beta=beta,
                     stop_condition=f'abs(z) > {stop_z} or cointegration_breaks(resid)',
                     targets=[(tp_z, 0.5, None), (0.5, 0.5, None)])   # z=0 then z=+/-0.5 remainder
```

F · RELATIVE VALUE (PAIRS)

# F2 Beta-Hedged Residual Reversion

A single instrument's residual, after hedging out its index beta, mean-reverts.

### VALID REGIME

`Rolling 250-bar OLS β stable, hedge R² > 0.6`

### GATE (ALL MUST HOLD)

- Rolling 250-bar OLS beta stable
- $R^2$ of the hedge > 0.6

### TRIGGER

Residual beyond ±2σ of its own 60-bar distribution after hedging β×index.

### ENTRY

Long/short the residual, hedged β× in the index, sized to the residual signal.

### STOP

Same z-logic as F1: ±3σ.

### TAKE-PROFIT / EXIT

Target 0 (fully mean-reverted residual).

### TRAIL / TIME STOP / KILL SWITCH

- Factor/sector play — only usable if you can source the hedge instrument's data; otherwise drop F2 and run F1

### PSEUDOCODE

```
def signal_F2(bars, index_bars, i):
    beta, r2 = rolling_ols_beta(bars, index_bars, i, 250)
    if r2 <= 0.6 or not beta_stable(beta, tol=0.15):
        return None

    residual = bars[i].close - beta*index_bars[i].close
    z = zscore_series(residual_series(bars, index_bars, beta, i), i, 60)
    if abs(z) <= 2.0:
        return None

    direction = 'long' if z < -2 else 'short'
    hedge_qty = beta * position_size(bars, i)

    return Order('F2', direction, 'market', bars[i].close,
                 stop=None, stop_condition='abs(z) > 3.0',
                 targets=[(0.0, 1.0, None)],
                 hedge_leg=('index', -hedge_qty if direction=='long' else hedge_qty))
```

G · TIME & CALENDAR

# G1 Session / Time-of-Day Bias

A structural intraday bias tied to a specific session window.

**VALID REGIME**

`Requires timestamped session definition, ≥ 2 years history`

**GATE (ALL MUST HOLD)**

- Session/timezone defined
- Average return + hit rate by hour-of-day computed, with t-stats
- Validated on a separate out-of-sample window

**TRIGGER**

Time-window opens, direction aligned with ADX/ER trend state (never take a session bias against the trend).

**ENTRY**

Market at window open, direction from the pre-computed bias.

**STOP**

1×ATR or the session’s opening range, whichever is tighter.

**TAKE-PROFIT / EXIT**

Fixed R target; hard flat at window close.

**TRAIL / TIME STOP / KILL SWITCH**

- Warning: the single most data-mined category in retail algo — freeze parameters, test only on untouched data, discard if the edge doesn’t survive

**PSEUDOCODE**

```
def signal_G1(bars, i, session, bias_table):
    hour = session_hour(bars, i, session)
    edge = bias_table.get(hour)                      # precomputed {hour: (mean_ret, hit_rate, t_stat)}
    if edge is None or edge.t_stat < 2.0 or not edge.out_of_sample_validated:
        return None
    if not aligned_with_trend(bars, i, edge.direction):
        return None
    if not at_window_open(bars, i, session, hour):
        return None

    atr_v = atr(bars, i, 14)
    or_range = opening_range_width(bars, session)
    entry = bars[i].close
    stop_dist = min(atr_v, or_range)
    stop = entry - stop_dist if edge.direction=='long' else entry + stop_dist
    tp = entry + 1.5*stop_dist if edge.direction=='long' else entry - 1.5*stop_dist

    return Order('G1', edge.direction, 'market', entry, stop,
                 targets=[(tp, 1.0, None)], hard_flat_at=session.window_close)
```

# G2 Seasonality (Turn-of-Month / Day-of-Week)

A calendar-anchored return bias (turn-of-month, day-of-week).

**VALID REGIME**

`≥ 3 years of data, ≥ 30 observations per bucket`

**GATE (ALL MUST HOLD)**

- ≥ 3 years of data
- Effect size checked against a bootstrap distribution
- ≥ 30 observations per calendar bucket

**TRIGGER**

Calendar rule fires (e.g. last trading day of month); enter at a fixed time window.

**ENTRY**

Market at the fixed calendar/time trigger.

**STOP**

1.5×ATR.

**TAKE-PROFIT / EXIT**

Fixed 1.5R; exit at a fixed date/time regardless of P&L.

**TRAIL / TIME STOP / KILL SWITCH**

- Reality check: expect Sharpe 0.3–0.6 at best — treat as a small satellite allocation, never core P&L

**PSEUDOCODE**

```
def signal_G2(bars, i, calendar_bucket, effect_table):
    bucket = calendar_bucket(bars, i)                  # e.g. 'turn_of_month', 'monday'
    edge = effect_table.get(bucket)
    if edge is None or edge.n_obs < 30 or not edge.significant_vs_bootstrap:
        return None
    if not calendar_trigger_fires(bars, i, bucket):
        return None

    atr_v = atr(bars, i, 14)
    entry = bars[i].close
    stop = entry - 1.5*atr_v if edge.direction=='long' else entry + 1.5*atr_v
    r = 1.5*atr_v
    tp = entry + 1.5*r if edge.direction=='long' else entry - 1.5*r

    return Order('G2', edge.direction, 'market', entry, stop,
                 targets=[(tp, 1.0, None)], hard_exit_at=edge.fixed_exit_datetime(bars, i))
```

H · ADAPTIVE RANGE LATTICE

# H1 ATR-Spaced Range Lattice

Laddered entries exploit a confirmed, non-trending chop range.

### VALID REGIME

`ADX < 18, Hurst < 0.45, range width ≥ 4×ATR`

### GATE (ALL MUST HOLD)

- ADX(14) < 18
- Hurst < 0.45
- Range width ≥ 4×ATR
- Regime monitor running every bar

### TRIGGER

Price crosses each 1×ATR-spaced ladder level inside the confirmed range.

### ENTRY

Laddered limits at range boundaries, 1×ATR spacing, max N = 5 tranches, each sized risk% / (2×ATR).

### STOP

Range boundary breached by 1×ATR → kill the entire ladder at once (never per-tranche).

### TAKE-PROFIT / EXIT

Each tranche exits at +1×ATR; overall inventory reduced as price crosses the mid.

### TRAIL / TIME STOP / KILL SWITCH

- Kill-switch (essential): ADX > 25 or ER > 0.3 → flatten everything within one bar
- Capital rule: reserve ≥ 60% of equity as unused margin at all times

### PSEUDOCODE

```
def signal_H1(bars, i, max_tranches=5):
    if not (adx(bars, i, 14) < 18 and hurst(bars, i, 256) < 0.45):
        return None
    rng_hi, rng_lo = confirmed_range(bars, i)
    atr_v = atr(bars, i, 14)
    if (rng_hi - rng_lo) < 4*atr_v:
        return None

    ladder = [rng_lo + k*atr_v for k in range(max_tranches)]
    orders = []
    for level in ladder:
        risk_pct = base_risk_pct() / max_tranches
        qty = (equity() * risk_pct) / (2*atr_v)
        orders.append(Order('H1', 'long', 'limit', level, level - 2*atr_v,
                            targets=[(level + atr_v, 1.0, None)], qty=qty))

    ladder_ctrl = Ladder(orders,
        kill_if=lambda b, j: (b[j].close < rng_lo - atr_v) or (b[j].close > rng_hi + atr_v)
                             or adx(b, j, 14) > 25 or efficiency_ratio(b, j, 20) > 0.3,
        reserve_margin_pct=60)
    return ladder_ctrl
```

S · CORE SIX (CLASSIC FORMULATION)

# S1 Trend + Pullback (EMA Stack + RSI)

A cleanly stacked EMA trend pulls back on RSI before resuming.

### VALID REGIME

`ADX(14) > 25`

### GATE (ALL MUST HOLD)

- ADX(14) > 25
- EMA(8) > EMA(21) > EMA(55) [reverse for short]

### TRIGGER

RSI(14) ∈ [40,55] (long) or C ≤ EMA21 + 0.3×ATR; then C_t > C_{t-1} or a bullish rejection candle.

### ENTRY

Market, on trigger-bar close (or next-bar open).

### STOP

min(swing stop, entry − 2.0×ATR) for longs — pick the farther of structure vs. volatility.

### TAKE-PROFIT / EXIT

Fixed R: entry + 2.0×(entry − stop) i.e. 2R, or entry + 3.0×ATR.

### TRAIL / TIME STOP / KILL SWITCH

- Trailing: move stop to breakeven at +1R, then trail 2.5×ATR or to new swing lows
- Hard exit on opposite EMA stack, or ADX < 20

### PSEUDOCODE

```
def signal_S1(bars, i):
    if adx(bars, i, 14) <= 25:
        return None
    e8, e21, e55 = ema(bars,i,8), ema(bars,i,21), ema(bars,i,55)
    long_stack, short_stack = e8 > e21 > e55, e8 < e21 < e55
    if not (long_stack or short_stack):
        return None
    direction = 'long' if long_stack else 'short'

    rsi_v, atr_v, c = rsi(bars, i, 14), atr(bars, i, 14), bars[i].close
    pullback = (40 <= rsi_v <= 55) if direction=='long' else (45 <= rsi_v <= 60)
    near_ema = (c <= e21 + 0.3*atr_v) if direction=='long' else (c >= e21 - 0.3*atr_v)
    if not (pullback or near_ema):
        return None
    trigger = (c > bars[i-1].close) or bullish_rejection(bars, i) if direction=='long'              else (c < bars[i-1].close) or
bearish_rejection(bars, i)
    if not trigger:
        return None

    sl_swing = swing_low(bars, i) if direction=='long' else swing_high(bars, i)
    sl_atr   = c - 2.0*atr_v if direction=='long' else c + 2.0*atr_v
    stop = min(sl_swing, sl_atr) if direction=='long' else max(sl_swing, sl_atr)
    r = abs(c - stop)
    tp = c + 2.0*r if direction=='long' else c - 2.0*r

    return Order('S1', direction, 'market', c, stop, targets=[(tp, 1.0, None)],
                 breakeven_at_R=1.0, trail='2.5ATR_or_new_swing',
                 hard_exit=['opposite_ema_stack', 'adx_below_20'])
```

S · CORE SIX (CLASSIC FORMULATION)

# S2 Volatility Breakout (Squeeze → Expansion)

Bollinger bandwidth compresses to a local minimum, then a Donchian break confirms expansion.

### VALID REGIME

`BW = min over lookback, or ATR < SMA(ATR,20)`

### GATE (ALL MUST HOLD)

- Bollinger Bandwidth(20,2) at N-bar minimum, or ATR < SMA(ATR,20)

### TRIGGER

C > max(H, 20-bar Donchian high); ATR_t > ATR_{t-1} (expansion confirms); optional bias C > EMA(55).

### ENTRY

Market, on the confirmed breakout bar.

### STOP

entry − 1.5×ATR, or breakout-bar low if tighter and still ≥ 1.2×ATR away.

### TAKE-PROFIT / EXIT

entry + 2.5×ATR (or 3×ATR); or Chandelier trail = max(H since entry) − 3×ATR.

### TRAIL / TIME STOP / KILL SWITCH

- Time stop: 8–12 bars — breakouts that stall usually fail

### PSEUDOCODE

```
def signal_S2(bars, i, N=20):
    bw = bollinger_bandwidth(bars, i, 20, 2)
    bw_hist = [bollinger_bandwidth(bars, j, 20, 2) for j in range(i-N, i)]
    squeeze = bw <= min(bw_hist) or atr(bars,i,14) < sma([atr(bars,j,14) for j in range(i-20,i)])
    if not squeeze:
        return None

    donchian_high = max(b.high for b in bars[i-20:i])
    c = bars[i].close
    atr_v, atr_prev = atr(bars, i, 14), atr(bars, i-1, 14)
    if not (c > donchian_high and atr_v > atr_prev):
        return None
    if c <= ema(bars, i, 55):
        return None                                    # optional directional bias filter

    stop_atr, stop_bar = c - 1.5*atr_v, bars[i].low
    stop = max(stop_atr, stop_bar) if (c - stop_bar) >= 1.2*atr_v else stop_atr
    tp = c + 2.5*atr_v

    return Order('S2', 'long', 'market', c, stop, targets=[(tp, 1.0, None)],
                 trail='chandelier_3ATR', time_stop_bars=10)
```

S · CORE SIX (CLASSIC FORMULATION)

# S3 Mean-Reversion in Range (RSI + Bollinger)

Oversold/overbought extreme inside a confirmed low-ADX range.

### VALID REGIME

`ADX(14) < 20`

### GATE (ALL MUST HOLD)

- ADX(14) < 20
- Price inside recent [swing_low, swing_high] range

### TRIGGER

RSI(14) < 30 (or RSI(2) < 10); low ≤ BB_lower; C_t > O_t bullish close.

### ENTRY

Market/close on the rejection bar.

### STOP

min(signal-bar low, BB_lower − 0.5×ATR), kept tight at 1.0–1.5×ATR.

### TAKE-PROFIT / EXIT

Primary: mid-Bollinger or mid-range. Secondary: opposite band, or 1.5–2R.

### TRAIL / TIME STOP / KILL SWITCH

- Immediate exit if ADX rises above 25 (regime change) or price closes outside the range

### PSEUDOCODE

```
def signal_S3(bars, i):
    if adx(bars, i, 14) >= 20:
        return None
    sh, sl = swing_high(bars, i), swing_low(bars, i)
    c, o, l = bars[i].close, bars[i].open, bars[i].low
    if not (sl <= c <= sh):
        return None

    rsi_v = rsi(bars, i, 14)
    bb_lower, bb_mid, bb_upper = bollinger_bands(bars, i, 20, 2)
    if not (rsi_v < 30 and l <= bb_lower and c > o):
        return None

    atr_v = atr(bars, i, 14)
    stop = min(l, bb_lower - 0.5*atr_v)

    order = Order('S3', 'long', 'market', c, stop,
                  targets=[(bb_mid, 0.6, None), (bb_upper, 0.4, None)])
    order.kill_if = lambda b, j: adx(b, j, 14) > 25 or b[j].close < sl or b[j].close > sh
    return order
```

S · CORE SIX (CLASSIC FORMULATION)

# S4 Pure Market Structure / Price Action

Classic HH-HL structure with a rejection-candle pullback entry.

### VALID REGIME

`Last two swings form HH + HL`

### GATE (ALL MUST HOLD)

- Structure state = HH/HL (bull)

### TRIGGER

Pullback to the last HL or EMA(21); confirmation candle: body/range > 0.6 and lower wick > 0.4×range; trigger = close > confirmation-bar high.

### ENTRY

Stop order above the confirmation bar high.

### STOP

Last swing low − 0.2×ATR (structure is the primary reference).

### TAKE-PROFIT / EXIT

Next swing high, or measured move = size of the prior impulse; or 2R–3R from the structure stop.

### TRAIL / TIME STOP / KILL SWITCH

- Trail stop to each new Higher Low once price makes a new Higher High

### PSEUDOCODE

```
def signal_S4(bars, i):
    if not structure_is('HH_HL', bars, i):
        return None

    last_hl, ema21, atr_v = last_higher_low(bars, i), ema(bars, i, 21), atr(bars, i, 14)
    c = bars[i].close
    pulled_back = abs(c - last_hl) < 0.5*atr_v or abs(c - ema21) < 0.5*atr_v
    if not pulled_back:
        return None

    body_ratio = abs(c - bars[i].open) / (bars[i].high - bars[i].low)
    lower_wick = (min(c, bars[i].open) - bars[i].low) / (bars[i].high - bars[i].low)
    if not (body_ratio > 0.6 and lower_wick > 0.4):
        return None

    entry = bars[i].high
    stop  = last_hl - 0.2*atr_v
    tp = entry + measured_move(bars, i)

    return Order('S4', 'long', 'stop', entry, stop, targets=[(tp, 1.0, None)],
                 trail='to_each_new_higher_low')
```

S · CORE SIX (CLASSIC FORMULATION)

# S5 Supertrend + ADX Filter

An ATR-based Supertrend flip, gated by a minimum ADX trend strength.

**VALID REGIME**

`ADX(14) > 25 at the flip`

**GATE (ALL MUST HOLD)**

- ADX(14) > 25

**TRIGGER**

Supertrend flips from Down to Up (or Up to Down for short).

**ENTRY**

Market, on the flip bar.

**STOP**

The Supertrend line itself, or Supertrend − 0.5×ATR buffer.

**TAKE-PROFIT / EXIT**

Exit on the opposite Supertrend flip; optional partial at 2×ATR, then trail the rest on the line.

**TRAIL / TIME STOP / KILL SWITCH**

- Disable new entries when ADX < 20

**PSEUDOCODE**

```
def signal_S5(bars, i, m=3.0, n=10):
    st, st_prev = supertrend(bars, i, m, n), supertrend(bars, i-1, m, n)
    flipped_up = st.direction == 'up' and st_prev.direction == 'down'
    if not (flipped_up and adx(bars, i, 14) > 25):
        return None

    c, atr_v = bars[i].close, atr(bars, i, 14)
    stop = st.value - 0.5*atr_v

    return Order('S5', 'long', 'market', c, stop,
                 targets=[(c + 2*atr_v, 0.5, 'take_partial'), (None, 0.5, 'trail_supertrend')],
                 hard_exit=['supertrend_flips_down'],
                 disable_new_entries_if='adx_below_20')
```

S · CORE SIX (CLASSIC FORMULATION)

# S6 Multi-Timeframe Momentum Confluence

A higher-timeframe trend bias times an execution-timeframe momentum trigger.

**VALID REGIME**

`HTF: C > EMA(50), structure HH-HL`

**GATE (ALL MUST HOLD)**

- HTF close > HTF EMA(50)
- HTF structure = HH/HL

**TRIGGER**

LTF RSI(14) crosses above 50 from below 40; LTF ATR > threshold; LTF close > LTF EMA(21).

**ENTRY**

Market, on the LTF trigger bar.

**STOP**

entry − 2.0×LTF ATR, or the most recent LTF swing low, whichever is farther.

**TAKE-PROFIT / EXIT**

2R–3R on the LTF, or LTF Supertrend/Chandelier trail.

**TRAIL / TIME STOP / KILL SWITCH**

- Exit immediately if the HTF bias flips

**PSEUDOCODE**

```
def signal_S6(bars_htf, bars_ltf, i_htf, i_ltf):
    htf_bias = bars_htf[i_htf].close > ema(bars_htf, i_htf, 50) \
               and structure_is('HH_HL', bars_htf, i_htf)
    if not htf_bias:
        return None

    rsi_now, rsi_prev = rsi(bars_ltf, i_ltf, 14), rsi(bars_ltf, i_ltf-1, 14)
    cross_up = rsi_prev < 40 and rsi_now > 50
    atr_ltf = atr(bars_ltf, i_ltf, 14)
    c = bars_ltf[i_ltf].close
    if not (cross_up and atr_ltf > atr_threshold(bars_ltf, i_ltf)
            and c > ema(bars_ltf, i_ltf, 21)):
        return None

    stop = min(c - 2.0*atr_ltf, swing_low(bars_ltf, i_ltf))
    r = c - stop
    tp = c + 2.5*r

    order = Order('S6', 'long', 'market', c, stop, targets=[(tp, 1.0, None)])
    order.kill_if = lambda bh, bl, jh, jl: bh[jh].close <= ema(bh, jh, 50)
    return order
```

# Best Time-Proven Strategies

A tiering of all 22, by how much genuine live/historical track record backs the underlying edge — not a prediction of future returns, and not investment advice. Use this to sequence validation: build and paper-test Tier 1 first.

### Tier 1 — Longest-standing, most independently documented edges

Classic trend-following breakout and pullback structures with decades of live evidence across managed futures / CTA books, plus the one reversal pattern (liquidity sweep) that is well documented in pure price data.

**B1 · S5 · A1 / S1 · D1**

### Tier 2 — Solid, regime-dependent edges

Real, published edges that only hold up when the regime gate is respected exactly — strong in-sample but more sensitive to parameter drift than Tier 1.

**B2 · A2 · C1 / S3 · E1**

### Tier 3 — Legitimate but narrower / situational

Sound logic, smaller or more conditional sample of favorable regimes. Good satellites once Tier 1 is validated, not a core allocation on their own.

**A3 · D2 · S4 · S6 · C2**

### Tier 4 — Highest fragility / infrastructure risk

G1/G2 are, by the blueprint's own warning, the most over-fitted category in retail algo trading — freeze parameters and test out-of-sample before trusting either. H1's grid/martingale-style structure has a long history of blowing up accounts without a hard, whole-ladder kill-switch. F1/F2 have genuine academic pedigree but need a second liquid instrument, borrow access, and real cointegration monitoring most retail infra doesn't have yet.

**G1 · G2 · H1 · F1 · F2**

---

Tiering reflects how well-attested the *underlying pattern* is in price-only trading literature and in this blueprint's own notes (e.g. the G-family's explicit Sharpe 0.3–0.6 ceiling, H1's mandatory kill-switch, D1 being called out as the best volume-free substitute for an order-flow read). It is not a backtest result on your data, and every strategy still needs its own walk-forward validation (Part 6 of the original blueprint) before being trusted with size. This document is for research and paper-trading education, not financial advice.